

Infografía

Módulo 1 – Introducción a Python

UPB · Gestión II/2025

Por qué Python?

- Sintaxis legible — cerca del pseudocódigo
- Ecosistema científico maduro (numpy, math)
- Es el lenguaje que usaremos para los primeros módulos:
rasterización, transformaciones, física
- Después saltaremos a **Godot** (GDScript), pero los conceptos serán los mismos

Lo que veremos hoy

1. Variables y tipos
2. Control de flujo (`if`, `while`, `for`)
3. Colecciones (`list`, `tuple`, `dict`)
4. Funciones
5. Clases y objetos

Cada concepto = un archivo en `1._intro_python/`.

1. Variables y tipos

```
edad = 19           # int
estatura = 1.9       # float
nombre = "juan"     # str
```

- No se declara el tipo — Python lo **infiere**
- `type(x)` te dice qué tipo tiene una variable
- Los tipos importan: no podés sumar `str + int` directamente

f-strings: la forma moderna

```
# antes:  
print("hola, soy " + nombre + ", tengo " + str(edad))  
  
# ahora:  
print(f"hola, soy {nombre}, tengo {edad}")
```

- Se evalúa cualquier expresión dentro de `{...}`
- No hay que convertir números a string a mano
- Se introdujo en Python 3.6 (2016)

Entrada del usuario

```
edad = float(input("ingrese edad: "))
```

`input()` siempre devuelve **string** — hay que convertir.

```
try:  
    edad = float(input("ingrese edad: "))  
except ValueError:  
    print("eso no es un numero")
```

`try/except` evita que el programa muera si el usuario escribe basura.

2. Control de flujo — `if`

```
if edad < 13:  
    print("niño")  
elif edad < 18:  
    print("adolescente")  
else:  
    print("adulto")
```

- **La indentación define el bloque** — no hay `{ }`
- 4 espacios, siempre

`while` — repetir mientras

```
contador = 5
while contador > 0:
    print(f"faltan {contador}...")
    contador -= 1
print("despegue!")
```

Se ejecuta **mientras la condición sea verdadera**.

Si nunca cambia la condición → bucle infinito.

`for` + `range` — repetir N veces

```
for i in range(5):  
    print(i)          # 0, 1, 2, 3, 4  
  
for n in range(0, 20, 2):  
    print(n)          # 0, 2, 4, ..., 18
```

`range(inicio, fin, paso)` — `fin` no incluido.

En gráficos vamos a iterar mucho: sobre píxeles, vértices, frames.

3. Colecciones — **list**

```
colores = ["rojo", "verde", "azul"]
colores.append("amarillo")
print(colores[0])      # "rojo"
print(colores[-1])     # "amarillo"
print(len(colores))    # 4
```

- Ordenadas, **mutables**
- Indexadas desde 0 (**[-1]** = último)

tuple — los puntos viven aquí

```
p1 = (100, 200)      # un punto (x, y)
print(p1[0], p1[1])  # 100 200
```

- Ordenadas, **inmutables** (no se pueden modificar)
- Perfectas para representar **puntos, colores, vectores**

```
poligono = [(0, 0), (100, 0), (100, 100), (0, 100)]
```

→ una lista de tuplas = los **vértices de un polígono**.

dict — pares clave-valor

```
color_rgb = {  
    "rojo": (255, 0, 0),  
    "verde": (0, 255, 0),  
    "azul": (0, 0, 255),  
}  
print(color_rgb["rojo"])    # (255, 0, 0)
```

- Acceso por **clave**, no por índice
- Útil para tablas de búsqueda — paletas, configuraciones, conteos

4. Funciones

```
def saludar(nombre, saludo="hola"):  
    print(f"{saludo}, {nombre}!")  
  
saludar("juan")  
saludar("ana", "buenos dias")
```

- `def` define la función
- Los parámetros pueden tener **valores por defecto**

Funciones que devuelven valores

```
import math

def distancia(p1, p2):
    dx = p2[0] - p1[0]
    dy = p2[1] - p1[1]
    return math.sqrt(dx * dx + dy * dy)

print(distancia((0, 0), (3, 4)))    # 5.0
```

Dos puntos como tuplas, una distancia como float.

Esto es geometría — y vamos a escribir muchas como esta.

Devolver varios valores

```
def punto_medio(p1, p2):  
    return (p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2  
  
mx, my = punto_medio((0, 0), (10, 20))
```

Python devuelve una **tupla**, y la podemos *desempacar* directamente.

5. Clases — agrupar datos + comportamiento

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        dx = other.x - self.x
        dy = other.y - self.y
        return math.sqrt(dx*dx + dy*dy)
```

`__init__` es el **constructor**. `self` = el objeto actual.

Usando la clase

```
a = Point(0, 0)
b = Point(3, 4)
print(a.distance_to(b))    # 5.0
```

- `Point(0, 0)` llama a `__init__`
- `a.distance_to(b)` pasa `a` como `self`, `b` como `other`

Una clase = un molde. Un objeto = una instancia del molde.

Por qué clases en gráficos?

Vamos a tener cosas como:

- `Polygon` con vértices, color, posición
- `Sprite` con imagen, posición, rotación
- `Tank` con cañón, balas, vida

Cada una **agrupa estado y comportamiento**. Una función suelta no alcanza.

Ejemplo completo: dado tramposo

```
class Dice:
    def __init__(self, n_sides, cheat_side, cheat_prob=0.3):
        self.n_sides = n_sides
        self.cheat_side = cheat_side
        self.cheat_prob = cheat_prob

    def throw(self):
        if random.random() <= self.cheat_prob:
            return self.cheat_side
        return random.randint(1, self.n_sides)
```

Verificando con Monte Carlo

```
dice = Dice(n_sides=7, cheat_side=3)

counts = {}
for _ in range(1000):
    value = dice.throw()
    counts[value] = counts.get(value, 0) + 1
```

`counts` = un `dict` que va contando cada resultado.
Mil tiradas → vemos que el `3` aparece mucho más de lo que debería.

Esto es simulación. Lo vamos a usar en física también.

Lo que sigue

- **Módulo 2** — un mini-juego por turnos: clases que se atacan entre sí
- **Módulo 3** — `arcade`: por fin pintamos píxeles
- **Módulo 3.1** — Bresenham: dibujar una línea **a mano**
- **Módulo 6** — transformaciones con matrices

Todo lo de hoy se va a usar. Las tuplas son puntos, los `dict` son tablas de búsqueda, las clases son objetos del juego.

Ejercicios

1. Modificá `dice2.py` para que el dado tenga **dos** caras tramposas
2. Escribí una función `perimetro(poligono)` que reciba una lista de tuplas
3. Convertí `Point` en `Point3D` (agregá `z` y actualizá `distance_to`)

Dejá los archivos en una carpeta `tarea_01/` y subilos al aula virtual.